

图 3-44 的右边画出了 s_1 、 s_2 和 p 指示的位置。图中还画出了 s_2 和 p 的值之间可能有一个偏移量为 e_2 字节的位置，该空间是未被使用的。数组 p 的结尾和 s_1 指示的位置之间还可能有一个偏移量为 e_1 字节的地方。

- 用数学语言解释第 5~7 行中计算 s_2 的逻辑。提示：想想 -16 的位级表示以及它在第 6 行 `andq` 指令中的作用。
- 用数学语言解释第 8~10 行中计算 p 的逻辑。提示：可以参考 2.3.7 节中有关除以 2 的幂的讨论。
- 对于下面 n 和 s_1 的值，跟踪代码的执行，确定 s_2 、 p 、 e_1 和 e_2 的结果值。

n	s_1	s_2	p	e_1	e_2
5	2 065				
6	2 064				

- 这段代码为 s_2 和 p 的值提供了什么样的对齐属性？

3.11 浮点代码

处理器的浮点体系结构包括多个方面，会影响对浮点数据操作的程序如何被映射到机器上，包括：

- 如何存储和访问浮点数值。通常是通过某种寄存器方式来完成。
- 对浮点数据操作的指令。
- 向函数传递浮点数参数和从函数返回浮点数结果的规则。
- 函数调用过程中保存寄存器的规则——例如，一些寄存器被指定为调用者保存，而其他的被指定为被调用者保存。

简要回顾历史会对理解 x86-64 的浮点体系结构有所帮助。1997 年出现了 Pentium/MMX，Intel 和 AMD 都引入了持续数代的媒体(media)指令，支持图形和图像处理。这些指令本意是允许多个操作以并行模式执行，称为单指令多数据或 SIMD(读作 sim-dee)。在这种模式中，对多个不同的数据并行执行同一个操作。近年来，这些扩展有了长足的发展。名字经过了一系列大的修改，从 MMX 到 SSE(Streaming SIMD Extension，流式 SIMD 扩展)，以及最新的 AVX(Advanced Vector Extension，高级向量扩展)。每一代中，都有一些不同的版本。每个扩展都是管理寄存器组中的数据，这些寄存器组在 MMX 中称为“MM”寄存器，SSE 中称为“XMM”寄存器，而在 AVX 中称为“YMM”寄存器；MM 寄存器是 64 位的，XMM 是 128 位的，而 YMM 是 256 位的。所以，每个 YMM 寄存器可以存放 8 个 32 位值，或 4 个 64 位值，这些值可以是整数，也可以是浮点数。

2000 年 Pentium 4 中引入了 SSE2，媒体指令开始包括那些对标量浮点数据进行操作的指令，使用 XMM 或 YMM 寄存器的低 32 位或 64 位中的单个值。这个标量模式提供了一组寄存器和指令，它们更类似于其他处理器支持浮点数的方式。所有能够执行 x86-64 代码的处理器都支持 SSE2 或更高的版本，因此 x86-64 浮点数是基于 SSE 或 AVX 的，包括传递过程参数和返回值的规则[77]。

我们的讲述基于 AVX2，即 AVX 的第二个版本，它是在 2013 年 Core i7 Haswell 处理器中引入的。当给定命令行参数 `-mavx2` 时，GCC 会生成 AVX2 代码。基于不同版本的 SSE 以及第一个版本的 AVX 的代码从概念上来说是的类似的，不过指令名和格式有所不同。我们只介绍用 GCC 编译浮点程序时会出现的那些指令。其中大部分是标量 AVX 指令，我

们也会说明对整个数据向量进行操作的指令出现的情况。后文中的网络旁注 OPT: SIMD 更全面地说明了如何利用 SSE 和 AVX 的 SIMD 功能读者可能希望参考 AMD 和 Intel 对每条指令的说明文档[4, 51]。和整数操作一样，注意我们表述中使用的 ATT 格式不同于这些文档中使用的 Intel 格式。特别地，这两种版本中列出指令操作数的顺序是不同的。

如图 3-45 所示，AVX 浮点体系结构允许数据存储在 16 个 YMM 寄存器中，它们的名字为 %ymm0~%ymm15。每个 YMM 寄存器都是 256 位(32 字节)。当对标量数据操作时，这些寄存器只保存浮点数，而且只使用低 32 位(对于 float)或 64 位(对于 double)。汇编代码用寄存器的 SSE XMM 寄存器名字 %xmm0~%xmm15 来引用它们，每个 XMM 寄存器都是对应的 YMM 寄存器的低 128 位(16 字节)。

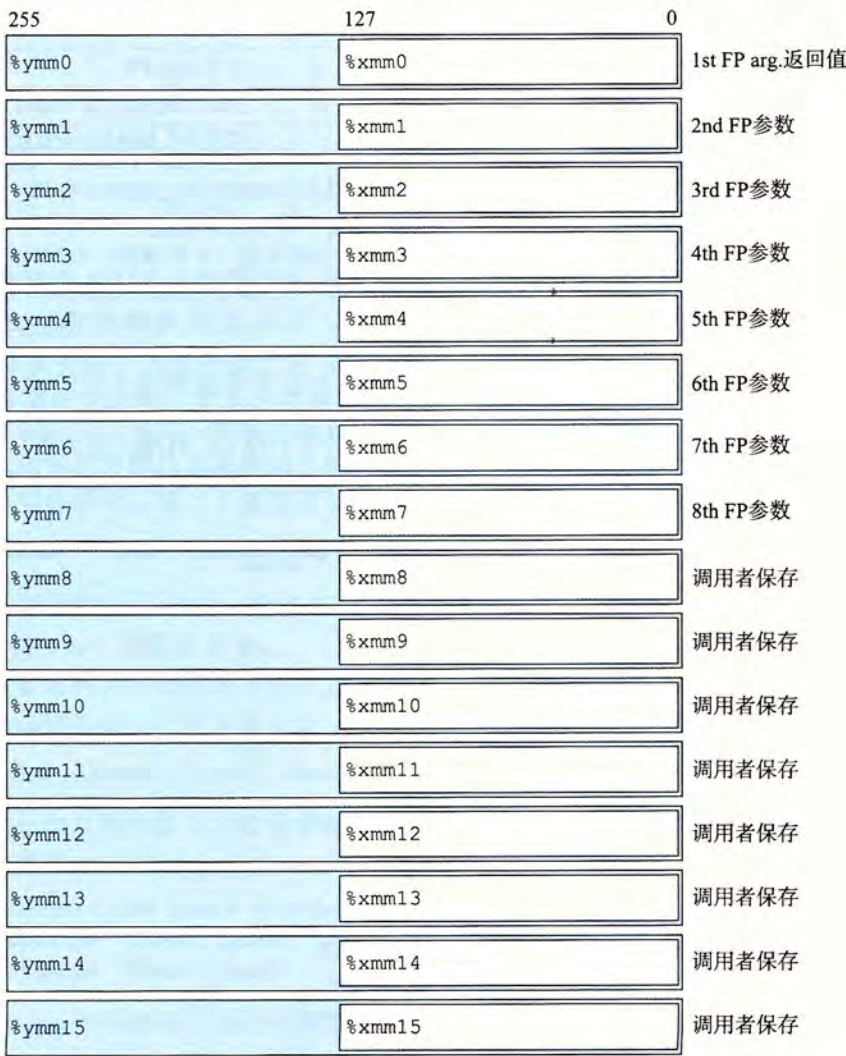


图 3-45 媒体寄存器。这些寄存器用于存放浮点数据。每个 YMM 寄存器保存 32 个字节。低 16 字节可以作为 XMM 寄存器来访问

3.11.1 浮点传送和转换操作

图 3-46 给出了一组在内存和 XMM 寄存器之间以及从一个 XMM 寄存器到另一个不

做任何转换的传送浮点数的指令。引用内存的指令是标量指令，意味着它们只对单个而不是一组封装好的数据值进行操作。数据要么保存在内存中(由表中的 M_{32} 和 M_{64} 指明)，要么保存在 XMM 寄存器中(在表中用 X 表示)。无论数据对齐与否，这些指令都能正确执行，不过代码优化规则建议 32 位内存数据满足 4 字节对齐，64 位数据满足 8 字节对齐。内存引用的指定方式与整数 MOV 指令的一样，包括偏移量、基址寄存器、变址寄存器和伸缩因子的所有可能的组合。

指令	源	目的	描述
vmovss	M_{32}	X	传送单精度数
vmovss	X	M_{32}	传送单精度数
vmovsd	M_{64}	X	传送双精度数
vmovsd	X	M_{64}	传送双精度数
vmovaps	X	X	传送对齐的封装好的单精度数
vmovapd	X	X	传送对齐的封装好的双精度数

图 3-46 浮点传送指令。这些操作在内存和寄存器之间以及一对寄存器之间传送值(X: XMM 寄存器(例如 %xmm3); M_{32} : 32 位内存范围; M_{64} : 64 位内存范围)

GCC 只用标量传送操作从内存传送数据到 XMM 寄存器或从 XMM 寄存器传送数据到内存。对于在两个 XMM 寄存器之间传送数据，GCC 会使用两种指令之一，即用 vmovaps 传送单精度数，用 vmovapd 传送双精度数。对于这些情况，程序复制整个寄存器还是只复制低位值既不会影响程序功能，也不会影响执行速度，所以使用这些指令还是针对标量数据的指令没有实质上的差别。指令名字中的字母‘a’表示“aligned(对齐的)”。当用于读写内存时，如果地址不满足 16 字节对齐，它们会导致异常。在两个寄存器之间传送数据，绝不会出现错误对齐的状况。

下面是一个不同浮点传送操作的例子，考虑以下 C 函数

```
float float_mov(float v1, float *src, float *dst) {
    float v2 = *src;
    *dst = v1;
    return v2;
}
```

与它相关联的 x86-64 汇编代码为

```
float float_mov(float v1, float *src, float *dst)
v1 in %xmm0, src in %rdi, dst in %rsi
1 float_mov:
2     vmovaps %xmm0, %xmm1      Copy v1
3     vmovss  (%rdi), %xmm0     Read v2 from src
4     vmovss  %xmm1, (%rsi)     Write v1 to dst
5     ret                      Return v2 in %xmm0
```

这个例子中可以看到它使用了 vmovaps 指令把数据从一个寄存器复制到另一个，使用了 vmovss 指令把数据从内存复制到 XMM 寄存器以及从 XMM 寄存器复制到内存。

图 3-47 和图 3-48 给出了在浮点数和整数数据类型之间以及不同浮点格式之间进行转换的指令集合。这些都是对单个数据值进行操作的标量指令。图 3-47 中的指令把一个从 XMM 寄存器或内存中读出的浮点值进行转换，并将结果写入一个通用寄存器(例如 %rax、%ebx 等)。把浮点值转换成整数时，指令会执行截断(truncation)，把值向 0 进行舍

人，这是 C 和大多数其他编程语言的要求。

指令	源	目的	描述
<code>vcvtts2si</code>	X/M_{32}	R_{32}	用截断的方法把单精度数转换成整数
<code>vcvttsd2si</code>	X/M_{64}	R_{32}	用截断的方法把双精度数转换成整数
<code>vcvtts2siq</code>	X/M_{32}	R_{64}	用截断的方法把单精度数转换成四字节整数
<code>vcvttsd2siq</code>	X/M_{64}	R_{64}	用截断的方法把双精度数转换成四字节整数

图 3-47 双操作数浮点转换指令。这些操作将浮点数转换成整数(X : XMM 寄存器(例如 `%xmm3`); R_{32} : 32 位通用寄存器(例如 `%eax`); R_{64} : 64 位通用寄存器(例如 `%rax`); M_{32} : 32 位内存范围; M_{64} : 64 位内存范围)

指令	源 1	源 2	目的	描述
<code>vcvttsi2ss</code>	M_{32}/R_{32}	X	X	把整数转换成单精度数
<code>vcvttsi2sd</code>	M_{32}/R_{32}	X	X	把整数转换成双精度数
<code>vcvttsi2ssq</code>	M_{64}/R_{64}	X	X	把四字节整数转换成单精度数
<code>vcvttsi2sdq</code>	M_{64}/R_{64}	X	X	把四字节整数转换成双精度数

图 3-48 三操作数浮点转换指令。这些操作将第一个源的数据类型转换成目的的数据类型。第二个源值对结果的低位字节没有影响(X : XMM 寄存器(例如 `%xmm3`); M_{32} : 32 位内存范围; M_{64} : 64 位内存范围)

图 3-48 中的指令把整数转换成浮点数。它们使用的是不太常见的三操作数格式，有两个源和一个目的。第一个操作数读自于内存或一个通用目的寄存器。这里可以忽略第二个操作数，因为它的值只会影响结果的高位字节。而我们的目标必须是 XMM 寄存器。在最常见的使用场景中，第二个源和目的操作数都是一样的，就像下面这条指令：

```
vcvttsi2sdq    %rax, %xmm1, %xmm1
```

这条指令从寄存器 `%rax` 读出一个长整数，把它转换成数据类型 `double`，并把结果存放在 XMM 寄存器 `%xmm1` 的低字节中。

最后，要在两种不同的浮点格式之间转换，GCC 的当前版本生成的代码需要单独说明。假设 `%xmm0` 的低位 4 字节保存着一个单精度值，很容易就想到用下面这条指令

```
vcvtss2sd     %xmm0, %xmm0, %xmm0
```

把它转换成一个双精度值，并将结果存储在寄存器 `%xmm0` 的低 8 字节。不过我们发现 GCC 生成的代码如下

```
Conversion from single to double precision
1  vunpcklps   %xmm0, %xmm0, %xmm0   Replicate first vector element
2  vcvtps2pd   %xmm0, %xmm0          Convert two vector elements to double
```

`vunpcklps` 指令通常用来交叉放置来自两个 XMM 寄存器的值，把它们存储到第三个寄存器中。也就是说，如果一个源寄存器的内容为字 $[s_3, s_2, s_1, s_0]$ ，另一个源寄存器为字 $[d_3, d_2, d_1, d_0]$ ，那么目的寄存器的值会是 $[s_1, d_1, s_0, d_0]$ 。在上面的代码中，我们看到三个操作数使用同一个寄存器，所以如果原始寄存器的值为 $[x_3, x_2, x_1, x_0]$ ，那么该指令会将寄存器的值更新为值 $[x_1, x_1, x_0, x_0]$ 。`vcvtps2pd` 指令把源 XMM 寄存器中的两个低位单精度值扩展成目的 XMM 寄存器中的两个双精度值。对前面 `vunpcklps` 指令的结果应用这条指令会得到值 $[dx_0, dx_0]$ ，这里 dx_0 是将 x 转换成双精度后的结果。

即，这两条指令的最终效果是将原始的%xmm0 低位 4 字节中的单精度值转换成双精度值，再将其两个副本保存到%xmm0 中。我们不太清楚 GCC 为什么会生成这样的代码，这样做既没有好处，也没有必要在 XMM 寄存器中把这个值复制一遍。

对于把双精度转换为单精度，GCC 会产生类似的代码：

```

Conversion from double to single precision
1  vmovddup    %xmm0, %xmm0      Replicate first vector element
2  vcvtpd2psx  %xmm0, %xmm0      Convert two vector elements to single

```

假设这些指令开始执行前寄存器%xmm0 保存着两个双精度值 $[x_1, x_0]$ 。然后 vmovddup 指令把它设置为 $[x_0, x_0]$ 。vcvtpd2psx 指令把这两个值转换成单精度，再存放到该寄存器的低位一半中，并将高位一半设置为 0，得到结果 $[0.0, 0.0, x_0, x_0]$ （回想一下，浮点值 0.0 是由位模式全 0 表示的）。同样，用这种方式把一种精度转换成另一种精度，而不用下面的单条指令，没有明显直接的意义：

```
vcvtss2sd %xmm0, %xmm0, %xmm0
```

下面是一个不同浮点转换操作的例子，考虑以下 C 函数

```

double fcvt(int i, float *fp, double *dp, long *lp)
{
    float f = *fp; double d = *dp; long l = *lp;
    *lp = (long) d;
    *fp = (float) i;
    *dp = (double) l;
    return (double) f;
}

```

以及它对应的 x86-64 汇编代码

```

double fcvt(int i, float *fp, double *dp, long *lp)
i in %edi, fp in %rsi, dp in %rdx, lp in %rcx
1  fcvt:
2      vmovss  (%rsi), %xmm0          Get f = *fp
3      movq    (%rcx), %rax           Get l = *lp
4      vcvttss2siq  (%rdx), %r8      Get d = *dp and convert to long
5      movq    %r8, (%rcx)           Store at lp
6      vcvtsi2ss   %edi, %xmm1, %xmm1  Convert i to float
7      vmovss  %xmm1, (%rsi)          Store at fp
8      vcvtsi2sdq   %rax, %xmm1, %xmm1  Convert l to double
9      vmovsd  %xmm1, (%rdx)          Store at dp
The following two instructions convert f to double
10     vunpcklps   %xmm0, %xmm0, %xmm0
11     vcvtps2pd   %xmm0, %xmm0
12     ret                                Return f

```

fcvt 的所有参数都是通过通用寄存器传递的，因为它们既不是整数也不是指针。结果通过寄存器%xmm0 返回。如图 3-45 中描述的，这是 float 或 double 值指定的返回寄存器。在这段代码中，可以看到图 3-46～图 3-48 中的许多传送和转换指令，还可以看到 GCC 将单精度转换为双精度的方法。

 **练习题 3.50** 对于下面的 C 代码，表达式 val1～val4 分别对应程序值 i、f、d 和 l：

```
double fcvt2(int *ip, float *fp, double *dp, long l)
```

```

{
    int i = *ip; float f = *fp; double d = *dp;
    *ip = (int)    val1;
    *fp = (float)  val2;
    *dp = (double) val3;
    return (double) val4;
}

```

根据该函数如下的 x86-64 代码，确定这个映射关系：

```

double fcvt2(int *ip, float *fp, double *dp, long l)
ip in %rdi, fp in %rsi, dp in %rdx, l in %rcx
Result returned in %xmm0
1  fcvt2:
2      movl    (%rdi), %eax
3      vmovss  (%rsi), %xmm0
4      vcvtsd2si    (%rdx), %r8d
5      movl    %r8d, (%rdi)
6      vcvtss2ss    %eax, %xmm1, %xmm1
7      vmovss  %xmm1, (%rsi)
8      vcvtss2sdq    %rcx, %xmm1, %xmm1
9      vmovsd  %xmm1, (%rdx)
10     vunpcklps    %xmm0, %xmm0, %xmm0
11     vcvtps2pd    %xmm0, %xmm0
12     ret

```



练习题 3.51 下面的 C 函数将类型为 `src_t` 的参数转换为类型为 `dst_t` 的返回值，这里两种数据类型都用 `typedef` 定义：

```

dest_t cvt(src_t x)
{
    dest_t y = (dest_t) x;
    return y;
}

```

在 x86-64 上执行这段代码，假设参数 `x` 在 `%xmm0` 中，或者在寄存器 `%rdi` 的某个适当的命名部分中（即 `%rdi` 或 `%edi`）。用一条或两条指令来完成类型转换，并把结果复制制到寄存器 `%rax` 的某个适当命名部分中（整数结果），或 `%xmm0` 中（浮点结果）。给出这条或这些指令，包括源和目的寄存器。

T_x	T_y	指令
long	double	<code>vcvtss2sdq %rdi,%xmm0</code>
double	int	
double	float	
long	float	
float	long	

3.11.2 过程中的浮点代码

在 x86-64 中，XMM 寄存器用来向函数传递浮点参数，以及从函数返回浮点值。如图 3-45 所示，可以看到如下规则：

- XMM 寄存器 `%xmm0` ~ `%xmm7` 最多可以传递 8 个浮点参数。按照参数列出的顺序使用这些寄存器。可以通过栈传递额外的浮点参数。

- 函数使用寄存器 %xmm0 来返回浮点值。
- 所有的 XMM 寄存器都是调用者保存的。被调用者可以不用保存就覆盖这些寄存器中任意一个。

当函数包含指针、整数和浮点数混合的参数时，指针和整数通过通用寄存器传递，而浮点值通过 XMM 寄存器传递。也就是说，参数到寄存器的映射取决于它们的类型和排列的顺序。下面是一些例子：

```
double f1(int x, double y, long z);
```

这个函数会把 x 存放在 %edi 中，y 放在 %xmm0 中，而 z 放在 %rsi 中。

```
double f2(double y, int x, long z);
```

这个函数的寄存器分配与函数 f1 相同。

```
double f1(float x, double *y, long *z);
```

这个函数会将 x 放在 %xmm0 中，y 放在 %rdi 中，而 z 放在 %rsi 中。

 **练习题 3.52** 对于下面每个函数声明，确定参数的寄存器分配：

- A. double g1(double a, long b, float c, int d);
- B. double g2(int a, double *b, float *c, long d);
- C. double g3(double *a, double b, int c, float d);
- D. double g4(float a, int *b, float c, double d);

3.11.3 浮点运算操作

图 3-49 描述了一组执行算术运算的标量 AVX2 浮点指令。每条指令有一个 (S_1) 或两个 (S_1, S_2) 源操作数，和一个目的操作数 D 。第一个源操作数 S_1 可以是一个 XMM 寄存器或一个内存位置。第二个源操作数和目的操作数都必须是 XMM 寄存器。每个操作都有一条针对单精度的指令和一条针对双精度的指令。结果存放在目的寄存器中。

单精度	双精度	效果	描述
vaddss	vaddsd	$D \leftarrow S_2 + S_1$	浮点数加
vsubss	vsubsd	$D \leftarrow S_2 - S_1$	浮点数减
vmulss	vmulsd	$D \leftarrow S_2 \times S_1$	浮点数乘
vdivss	vdivsd	$D \leftarrow S_2 / S_1$	浮点数除
vmaxss	vmaxsd	$D \leftarrow \max(S_2, S_1)$	浮点数最大值
vminss	vminsd	$D \leftarrow \min(S_2, S_1)$	浮点数最小值
sqrtps	sqrtsd	$D \leftarrow \sqrt{S_1}$	浮点数平方根

图 3-49 标量浮点算术运算。这些指令有一个或两个源操作数和一个目的操作数

来看一个例子，考虑下面的浮点函数：

```
double funct(double a, float x, double b, int i)
{
    return a*x - b/i;
}
```

x86-64 代码如下：

```

double funct(double a, float x, double b, int i)
a in %xmm0, x in %xmm1, b in %xmm2, i in %edi
1  funct:
    The following two instructions convert x to double
2  vunpcklps    %xmm1, %xmm1, %xmm1
3  vcvtps2pd    %xmm1, %xmm1
4  vmulsd      %xmm0, %xmm1, %xmm0      Multiply a by x
5  vcvtsi2sd    %edi, %xmm1, %xmm1      Convert i to double
6  vdivsd      %xmm1, %xmm2, %xmm2      Compute b/i
7  vsubsd      %xmm2, %xmm0, %xmm0      Subtract from a*x
8  ret                          Return

```

三个浮点参数 a 、 x 和 b 通过 XMM 寄存器 $\%xmm0 \sim \%xmm2$ 传递，而整数参数通过寄存器 $\%edi$ 传递。标准的双指令序列用以将参数 x 转换为双精度类型(第 2~3 行)。另一条转换指令用来将参数 i 转换为双精度类型(第 5 行)。该函数的值通过寄存器 $\%xmm0$ 返回。

 **练习题 3.53** 对于下面的 C 函数，4 个参数的类型由 typedef 定义：

```

double funct1(arg1_t p, arg2_t q, arg3_t r, arg4_t s)
{
    return p/(q+r) - s;
}

```

编译时，GCC 产生如下代码：

```

double funct1(arg1_t p, arg2_t q, arg3_t r, arg4_t s)
1  funct1:
2  vcvtsi2ssq    %rsi, %xmm2, %xmm2
3  vaddss       %xmm0, %xmm2, %xmm0
4  vcvtsi2ss     %edi, %xmm2, %xmm2
5  vdivss       %xmm0, %xmm2, %xmm0
6  vunpcklps    %xmm0, %xmm0, %xmm0
7  vcvtps2pd    %xmm0, %xmm0
8  vsubsd       %xmm1, %xmm0, %xmm0
9  ret

```

确定 4 个参数类型可能的组合(答案可能不止一种)。

 **练习题 3.54** 函数 `funct2` 具有如下原型：

```
double funct2(double w, int x, float y, long z);
```

GCC 为该函数产生如下代码：

```

double funct2(double w, int x, float y, long z)
w in %xmm0, x in %edi, y in %xmm1, z in %rsi
1  funct2:
2  vcvtsi2ss     %edi, %xmm2, %xmm2
3  vmulss       %xmm1, %xmm2, %xmm1
4  vunpcklps    %xmm1, %xmm1, %xmm1
5  vcvtps2pd    %xmm1, %xmm2
6  vcvtsi2sdq    %rsi, %xmm1, %xmm1
7  vdivsd       %xmm1, %xmm0, %xmm0
8  vsubsd       %xmm0, %xmm2, %xmm0
9  ret

```

写出 `funct2` 的 C 语言版本。

3. 11. 4 定义和使用浮点常数

和整数运算操作不同，AVX 浮点操作不能以立即数值作为操作数。相反，编译器必须为所有的常量值分配和初始化存储空间。然后代码在把这些值从内存读入。下面从摄氏度到华氏度转换的函数就说明了这个问题：

```
double cel2fahr(double temp)
{
    return 1.8 * temp + 32.0;
}
```

相应的 x86-64 汇编代码部分如下：

```
double cel2fahr(double temp)
temp in %xmm0
1 cel2fahr:
2 vmulsd .LC2(%rip), %xmm0, %xmm0    Multiply by 1.8
3 vaddsd .LC3(%rip), %xmm0, %xmm0    Add 32.0
4 ret
5 .LC2:
6 .long 3435973837                    Low-order 4 bytes of 1.8
7 .long 1073532108                    High-order 4 bytes of 1.8
8 .LC3:
9 .long 0                             Low-order 4 bytes of 32.0
10 .long 1077936128                    High-order 4 bytes of 32.0
```

可以看到函数从标号为.LC2 的内存位置读出值 1.8，从标号为.LC3 的位置读入值 32.0。观察这些标号对应的值，可以看出每一个都是通过一对.long 声明和十进制表示的值指定的。该怎样把这些数解释为浮点值呢？看看标号为.LC2 的声明，有两个值：3435973837 (0xcccccccd)和 1073532108(0x3ffcccccc)。因为机器采用的是小端法字节顺序，第一个值给出的是低位 4 字节，第二个给出的是高位 4 字节。从高位字节，可以抽取指数字段为 0x3ff(1023)，减去偏移 1023 得到指数 0。将两个值的小数位连接起来，得到小数字段 0xcccccccccccd，二进制小数表示为 0.8，加上隐含的 1 得到 1.8。

 **练习题 3.55** 解释标号为.LC3 处声明的数字是如何对数字 32.0 编码的。

3. 11. 5 在浮点代码中使用位级操作

有时，我们会发现 GCC 生成的代码会在 XMM 寄存器上执行位级操作，得到有用的浮点结果。图 3-50 展示了一些相关的指令，类似于它们在通用寄存器上对应的操作。这些操作都作用于封装好的数据，即它们更新整个目的 XMM 寄存器，对两个源寄存器的所有位都实施指定的位级操作。和前面一样，我们只对标量数据感兴趣，只想了解这些指令对目的寄存器的低 4 或 8 字节的影响。从下面的例子中可以看出，运用这些操作通常可以简单方便地操作浮点数。

单精度	双精度	效果	描述
vxorps	vorpd	$D \leftarrow S_2 \sim S_1$	位级异或(EXCLUSIVE-OR)
vandps	andpd	$D \leftarrow S_2 \& S_1$	位级与(AND)

图 3-50 对封装数据的位级操作(这些指令对一个 XMM 寄存器中的所有 128 位进行布尔操作)

 **练习题 3.56** 考虑下面的 C 函数，其中 `EXPR` 是用 `# define` 定义的宏：

```
double simplefun(double x) {
    return EXPR(x);
}
```

下面，我们给出了为不同的 `EXPR` 定义生成的 AVX2 代码，其中，`x` 的值保存在 `%xmm0` 中。这些代码都对应于某些对浮点数值有用的操作。确定这些操作都是什么。要理解从内存中取出的常数字的位模式才能找出答案。

```
A. 1    vmovsd    .LC1(%rip), %xmm1
    2    vandpd  %xmm1, %xmm0, %xmm0
    3    .LC1:
    4    .long    4294967295
    5    .long    2147483647
    6    .long    0
    7    .long    0

B. 1    vxorpd  %xmm0, %xmm0, %xmm0

C. 1    vmovsd  .LC2(%rip), %xmm1
    2    vxorpd  %xmm1, %xmm0, %xmm0
    3    .LC2:
    4    .long    0
    5    .long    -2147483648
    6    .long    0
    7    .long    0
```

3.11.6 浮点比较操作

AVX2 提供了两条用于比较浮点数值的指令：

指令	基于	描述
<code>ucomiss S₁, S₂</code>	$S_2 - S_1$	比较单精度值
<code>ucomisd S₁, S₂</code>	$S_2 - S_1$	比较双精度值

这些指令类似于 `CMP` 指令(参见 3.6 节)，它们都比较操作数 S_1 和 S_2 (但是顺序可能与预计的相反)，并且设置条件码指示它们的相对值。与 `cmpq` 一样，它们遵循以相反顺序列出操作数的 ATT 格式惯例。参数 S_2 必须在 XMM 寄存器中，而 S_1 可以在 XMM 寄存器中，也可以在内存中。

浮点比较指令会设置三个条件码：零标志位 `ZF`、进位标志位 `CF` 和奇偶标志位 `PF`。3.6.1 节中我们没有讲奇偶标志位，因为它在 GCC 产生的 x86 代码中不太常见。对于整数操作，当最近的一次算术或逻辑运算产生的值的最低位字节是偶校验的(即这个字节中有偶数个 1)，那么就会设置这个标志位。不过对于浮点比较，当两个操作数中任一个是 `NaN` 时，会设置该位。根据惯例，C 语言中如果有个参数为 `NaN`，就认为比较失败了，这个标志位就被用来发现这样的条件。例如，当 `x` 为 `NaN` 时，比较 `x==x` 都会得到 0。

条件码的设置条件如下：

顺序 $S_2 : S_1$	CF	ZF	PF
无序的	1	1	1
$S_2 < S_1$	1	0	0
$S_2 = S_1$	0	1	0
$S_2 > S_1$	0	0	0

当任一操作数为 NaN 时,就会出现无序的情况。可以通过奇偶标志位发现这种情况。通常 jp(jump on parity)指令是条件跳转,条件就是浮点比较得到一个无序的结果。除了这种情况以外,进位和零标志位的值都和对应的无符号比较一样:当两个操作数相等时,设置 ZF;当 $S_2 < S_1$ 时,设置 CF。像 ja 和 jb 这样的指令可以根据标志位的各种组合进行条件跳转。

来看一个浮点比较的例子,图 3-51a 中的 C 函数会根据参数 x 与 0.0 的相对关系进行分类,返回一个枚举类型作为结果。C 中的枚举类型是编码为整数的,所以函数可能的值为:0(NEG),1(ZERO),2(POS)和 3(OTHER)。当 x 的值为 NaN 时,会出现最后一种结果。

```
typedef enum {NEG, ZERO, POS, OTHER} range_t;

range_t find_range(float x)
{
    int result;
    if (x < 0)
        result = NEG;
    else if (x == 0)
        result = ZERO;
    else if (x > 0)
        result = POS;
    else
        result = OTHER;
    return result;
}
```

a) C 代码

```
range_t find_range(float x)
x in %xmm0
1  find_range:
2  vxorps %xmm1, %xmm1, %xmm1      Set %xmm1 = 0
3  vucomiss    %xmm0, %xmm1        Compare 0:x
4  ja         .L5                  If >, goto neg
5  vucomiss    %xmm1, %xmm0        Compare x:0
6  jp         .L8                  If NaN, goto posornan
7  movl      $1, %eax              result = ZERO
8  je         .L3                  If =, goto done
9  .L8:                                posornan:
10 vucomiss    .LC0(%rip), %xmm0    Compare x:0
11 setbe     %al                    Set result = NaN ? 1 : 0
12 movzbl    %al, %eax              Zero-extend
13 addl      $2, %eax               result += 2 (POS for > 0, OTHER for NaN)
14 ret                                Return
15 .L5:                                neg:
16 movl      $0, %eax              result = NEG
17 .L3:                                done:
18 rep; ret                          Return
```

b) 产生的汇编代码

图 3-51 浮点代码中的条件分支说明

GCC 为 `find_range` 生成图 3-51b 中的代码。这段代码的效率不是很高：它比较了 `x` 和 0.0 三次，即使一次比较就能获得所需的信息。它还生成了浮点常数两次：一次使用 `vxorps`，另一次从内存读出这个值。让我们追踪这个函数，看看四种可能的比较结果：

`x < 0.0` 第 4 行的 `ja` 分支指令会选择跳转，跳转到结尾，返回值为 0。

`x = 0.0` `ja` (第 4 行) 和 `jp` (第 6 行) 两个分支语句都会选择不跳转，但是 `je` 分支 (第 8 行) 会选择跳转，以 `%eax` 等于 1 返回。

`x > 0.0` 这三个分支都不会选择跳转。`setbe` (第 11 行) 会得到 0，`addl` 指令 (第 13 行) 会把它增加，得到返回值 2。

`x = NaN` `jp` 分支 (第 6 行) 会选择跳转。第三个 `vucomiss` 指令 (第 10 行) 会设置进位和零标志位，因此 `setbe` 指令 (第 11 行) 和后面的指令会把 `%eax` 设置为 1。`addl` 指令 (第 13 行) 会把它增加，得到返回值 3。

家庭作业 3.73 和 3.74 中，你需要试着手动生成 `find_range` 更高效的实现。

 **练习题 3.57** 函数 `funct3` 有如下原型：

```
double funct3(int *ap, double b, long c, float *dp);
```

对于此函数，GCC 产生如下代码：

```
double funct3(int *ap, double b, long c, float *dp)
ap in %rdi, b in %xmm0, c in %rsi, dp in %rdx
1  funct3:
2      vmovss (%rdx), %xmm1
3      vcvtsi2sd (%rdi), %xmm2, %xmm2
4      vucomisd %xmm2, %xmm0
5      jbe .L8
6      vcvtsi2ssq %rsi, %xmm0, %xmm0
7      vmulss %xmm1, %xmm0, %xmm1
8      vunpcklps %xmm1, %xmm1, %xmm1
9      vcvtps2pd %xmm1, %xmm0
10     ret
11     .L8:
12     vaddss %xmm1, %xmm1, %xmm1
13     vcvtsi2ssq %rsi, %xmm0, %xmm0
14     vaddss %xmm1, %xmm0, %xmm0
15     vunpcklps %xmm0, %xmm0, %xmm0
16     vcvtps2pd %xmm0, %xmm0
17     ret
```

写出 `funct3` 的 C 版本。

3.11.7 对浮点代码的观察结论

我们可以看到，用 AVX2 为浮点数上的操作产生的机器代码风格类似于为整数上的操作产生的代码风格。它们都使用一组寄存器来保存和操作数据值，也都使用这些寄存器来传递函数参数。

当然，处理不同的数据类型以及对包含混合数据类型的表达式求值的规则有许多复杂之处，同时，AVX2 代码包括许多比只执行整数运算的函数更加不同的指令和格式。

AVX2 还有能力在封装好的数据上执行并行操作，使计算执行得更快。编译器开发者正致力于自动化从标量代码到并行代码的转换，但是目前通过并行化获得更高性能的最可

靠的方法是使用 GCC 支持的、操纵向量数据的 C 语言扩展。参见原书 546 页的网络旁注 OPT: SIMD, 看看可以怎么做到这样。

3.12 小结

在本章中, 我们窥视了 C 语言提供的抽象层下面的东西, 以了解机器级编程。通过让编译器产生机器级程序的汇编代码表示, 我们了解了编译器和它的优化能力, 以及机器、数据类型和指令集。在第 5 章, 我们会看到, 当编写能有效映射到机器上的程序时, 了解编译器的特性会有所帮助。我们还更完整地了解了程序如何将数据存储在不同的内存区域中。在第 12 章会看到许多这样的例子, 应用程序员需要知道一个程序变量是在运行时栈中, 是在某个动态分配的数据结构中, 还是全局程序数据的一部分。理解程序如何映射到机器上, 会让理解这些存储类型之间的区别容易一些。

机器级程序和它们的汇编代码表示, 与 C 程序的差别很大。各种数据类型之间的差别很小。程序是以指令序列来表示的, 每条指令都完成一个单独的操作。部分程序状态, 如寄存器和运行时栈, 对程序员来说是直接可见的。本书仅提供了低级操作来支持数据处理和程序控制。编译器必须使用多条指令来产生和操作各种数据结构, 以及实现像条件、循环和过程这样的控制结构。我们讲述了 C 语言和如何编译它的许多不同方面。我们看到 C 语言中缺乏边界检查, 使得许多程序容易出现缓冲区溢出。虽然最近的运行时系统提供了安全保护, 而且编译器帮助使得程序更安全, 但是这已经使许多系统容易受到恶意入侵者的攻击。

我们只分析了 C 到 x86-64 的映射, 但是大多数内容对其他语言和机器组合来说也是类似的。例如, 编译 C++ 与编译 C 就非常相似。实际上, C++ 的早期实现就只是简单地执行了从 C++ 到 C 的源到源的转换, 并对结果运行 C 编译器, 产生目标代码。C++ 的对象用结构来表示, 类似于 C 的 struct。C++ 的方法是用指向实现方法的代码的指针来表示的。相比而言, Java 的实现方式完全不同。Java 的目标代码是一种特殊的二进制表示, 称为 Java 字节代码。这种代码可以看成是虚拟机的机器级程序。正如它的名字暗示的那样, 这种机器并不是直接用硬件实现的, 而是用软件解释器处理字节代码, 模拟虚拟机的行为。另外, 有一种称为及时编译(just-in-time compilation)的方法, 动态地将字节代码序列翻译成机器指令。当代码要执行多次时(例如在循环中), 这种方法执行起来更快。用字节代码作为程序的低级表示, 优点是相同的代码可以在许多不同的机器上执行, 而在本章谈到的机器代码只能在 x86-64 机器上运行。

参考文献说明

Intel 和 AMD 提供了关于他们处理器的大量文档。包括从汇编语言程序员角度来看硬件的概貌[2, 50], 还包括每条指令的详细参考[3, 51]。读指令描述很复杂, 因为 1)所有的文档都基于 Intel 汇编代码格式, 2)由于不同的寻址和执行模式, 每条指令都有多个变种, 3)没有说明性示例。不过这些文档仍然是关于每条指令行为的权威参考。

组织 x86-64.org 负责定义运行在 Linux 系统上的 x86-64 代码的应用二进制接口(Application Binary Interface, ABI)[77]。这个接口描述了一些细节, 包括过程链接、二进制代码文件和大量的为了让机器代码程序正确运行所需要的其他特性。

正如我们讨论过的那样, GCC 使用的 ATT 格式与 Intel 文档中使用的 Intel 格式和其他编译器(包括 Microsoft 编译器)使用的格式都很不相同。

Muchnick 的关于编译器设计的书[80]被认为是关于代码优化技术最全面的参考书。它涵盖了许多我们在此讨论过的技术, 例如寄存器使用规则。

已经有很多文章是关于使用缓冲区溢出通过因特网来攻击系统的。Spafford 出版了关于 1988 年因特网蠕虫的详细分析[105], 而帮助阻止它传播的 MIT 团队的成员也出版了一些论著[35]。从那以后, 大量的论文和项目提出了各种创建和阻止缓冲区溢出攻击的方法。Seacord 的书[97]提供了关于缓冲区溢出和其他一些对 C 编译器产生的代码进行攻击的丰富信息。

家庭作业

- 3.58 一个函数的原型为